



Artificial Intelligence in Interactive Systems

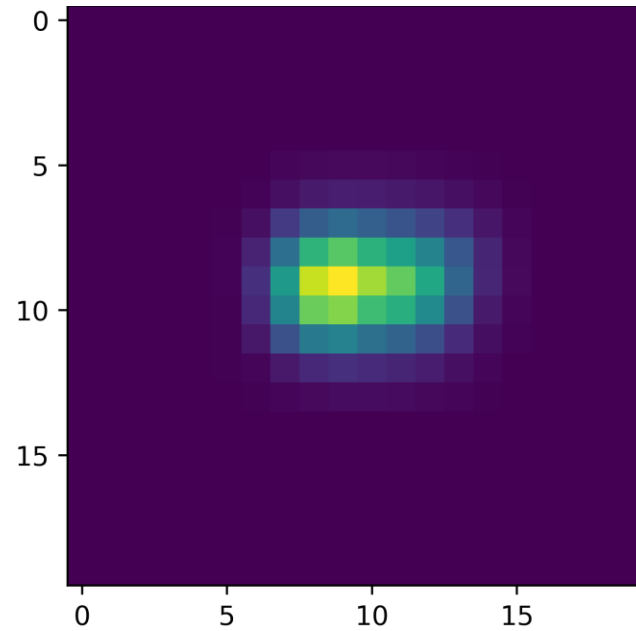
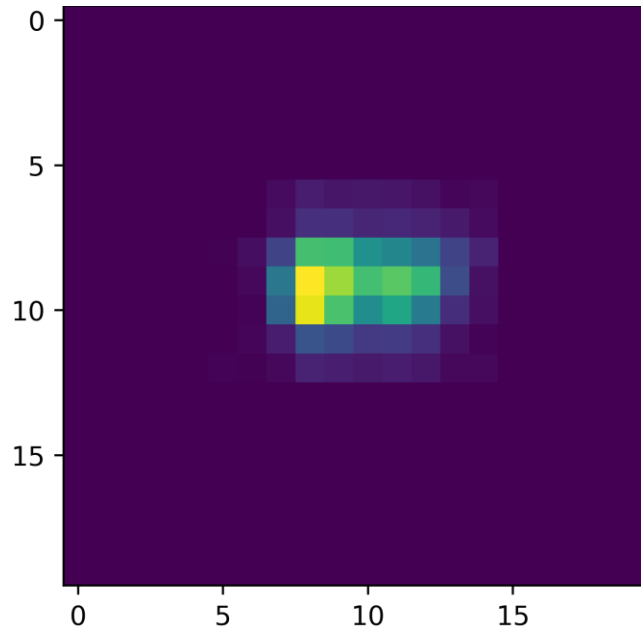
Additional Layers

Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN)

Filter

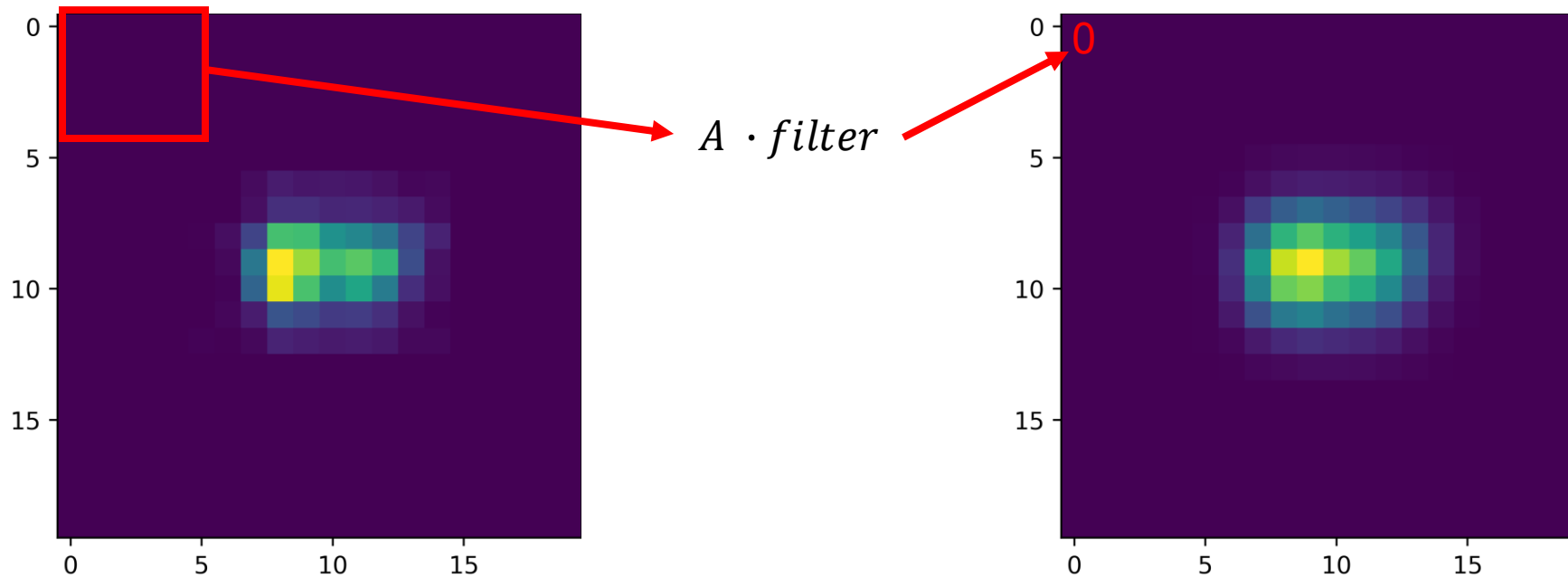
$$filter = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$



Convolutional Neural Network (CNN)

Process

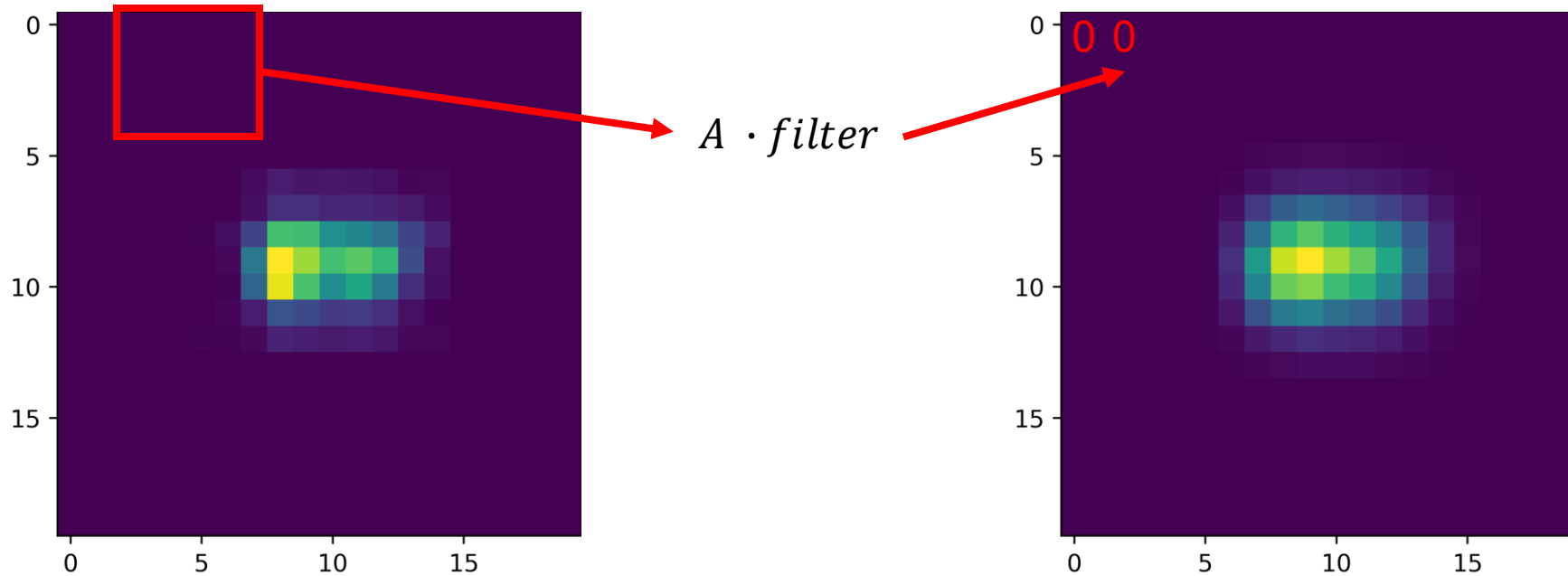
$$\text{filter} = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$



Convolutional Neural Network (CNN)

Process

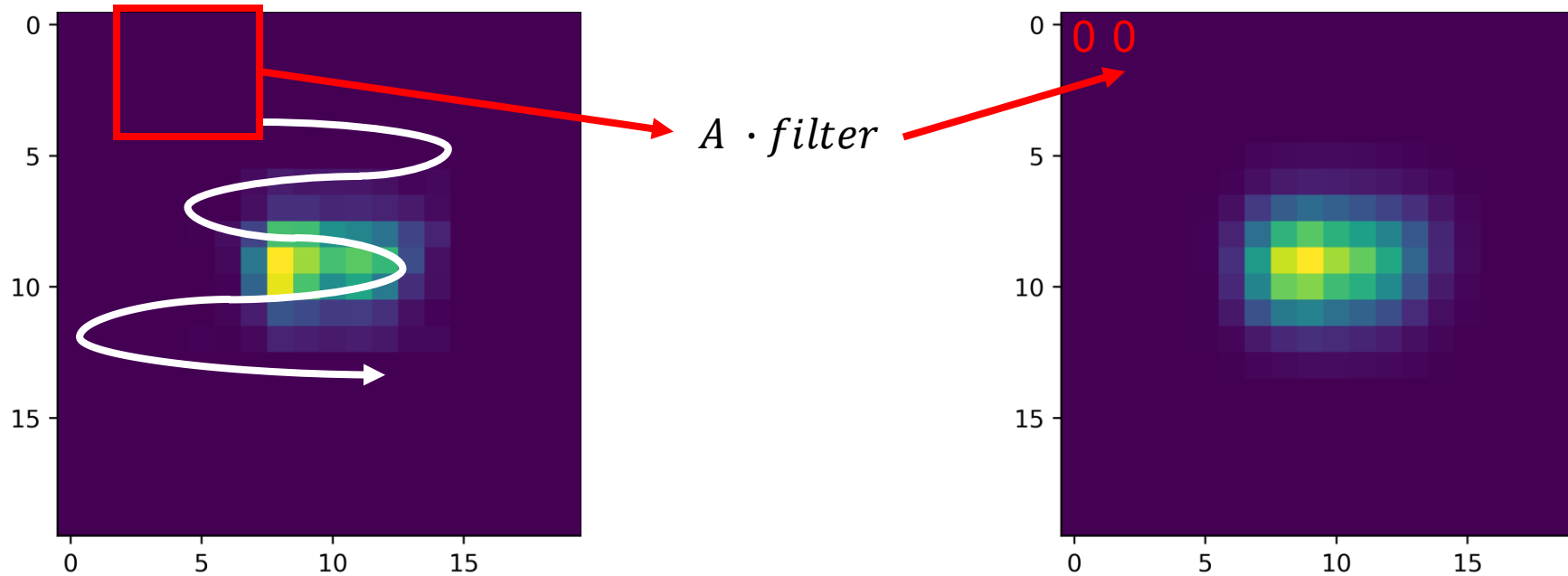
$$\text{filter} = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$



Convolutional Neural Network (CNN)

Process

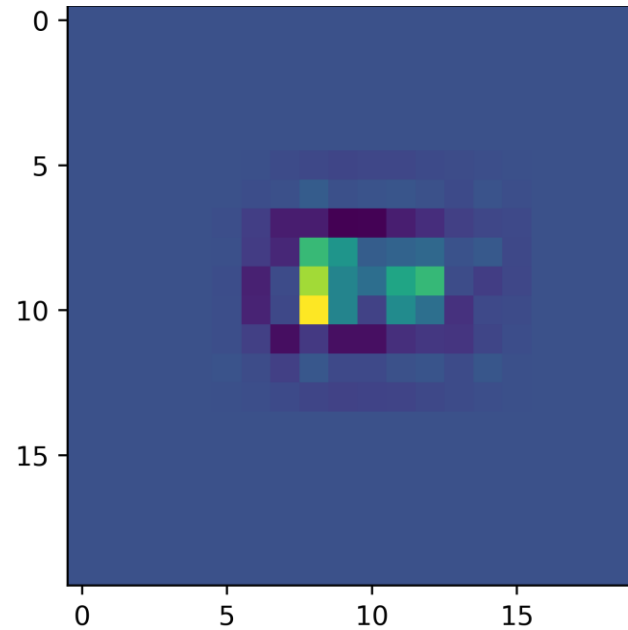
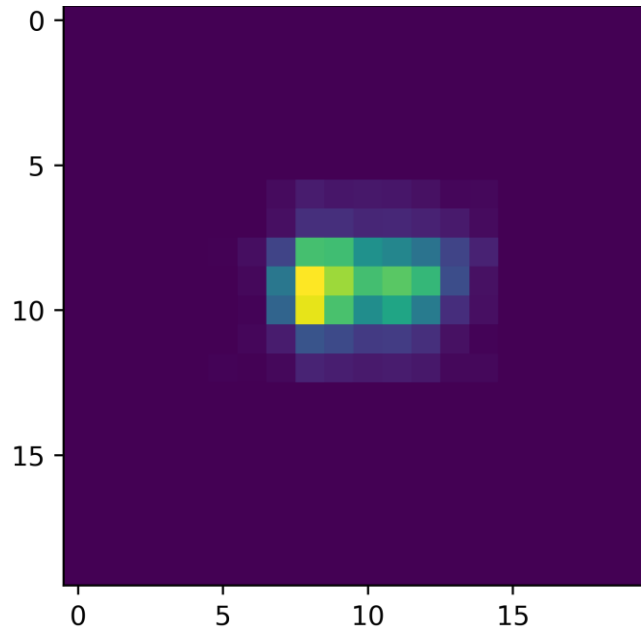
$$filter = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$



Convolutional Neural Network (CNN)

Filter – Edge Detection Example

$$filter = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Convolutional Neural Network (CNN)

`tf.keras.layers.Conv2D(filters, kernel_size, strides=(1, 1), padding='valid', ....)`

- Filter count
- Kernel size e.g. (3x3)
- Step length (strides) e.g. (1,1)
- Padding

$$filter = \begin{bmatrix} w_{0,0} & \cdots & w_{0,m} \\ \vdots & \ddots & \vdots \\ w_{n,0} & \cdots & w_{n,m} \end{bmatrix}$$

During the learning the model learns the w 's of each filter.

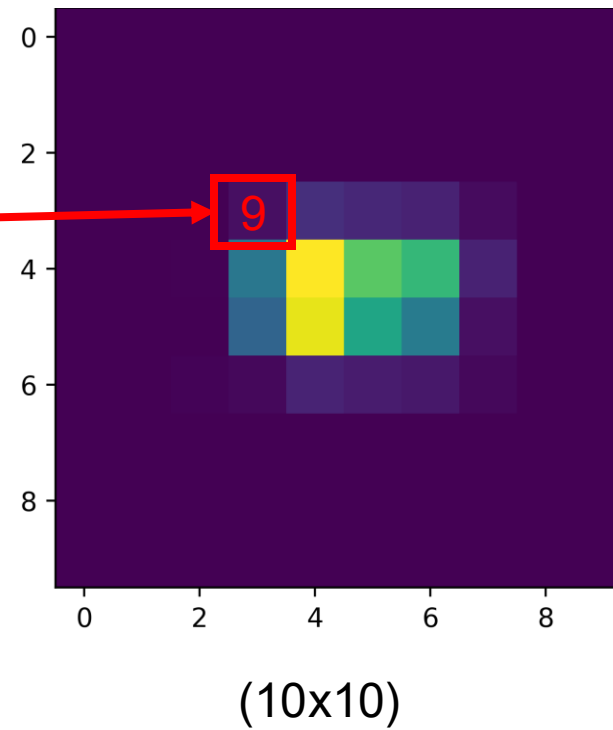
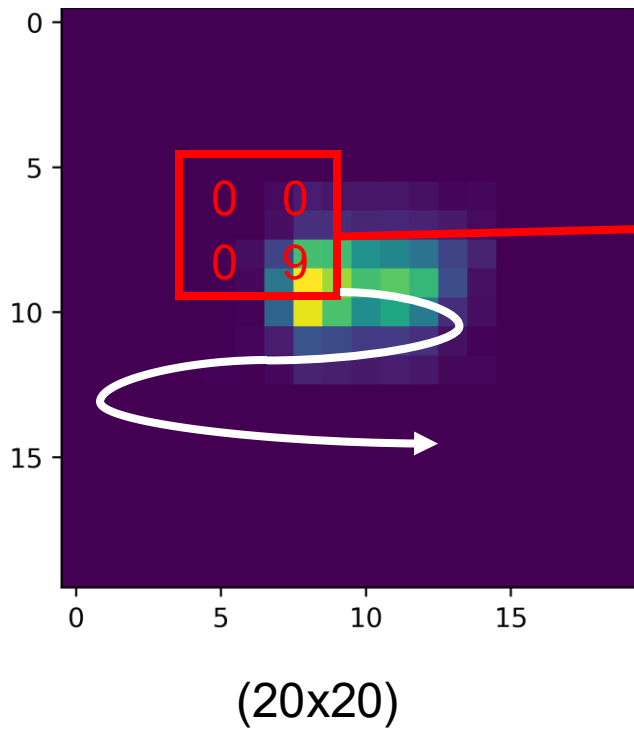
Constraints:

- Shaped Input
- Shaped Output

Pooling Layer

Max Pooling Layer

- Kernel size e.g. (2x2)
- Max value within the filter

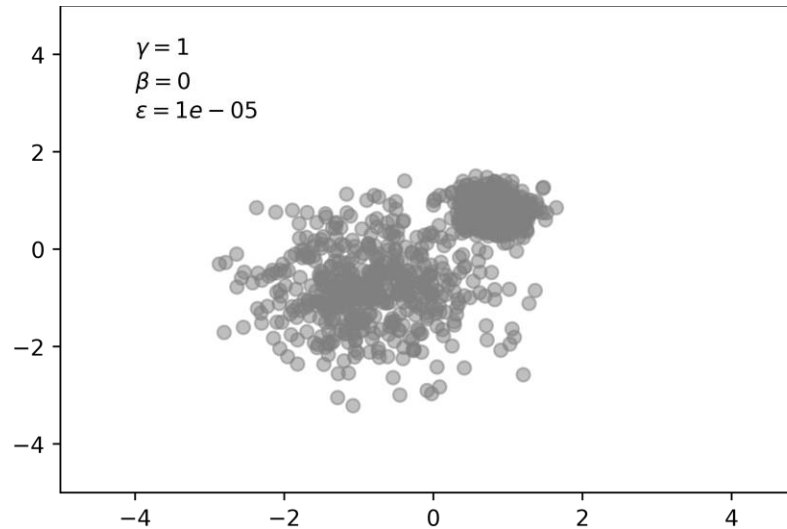
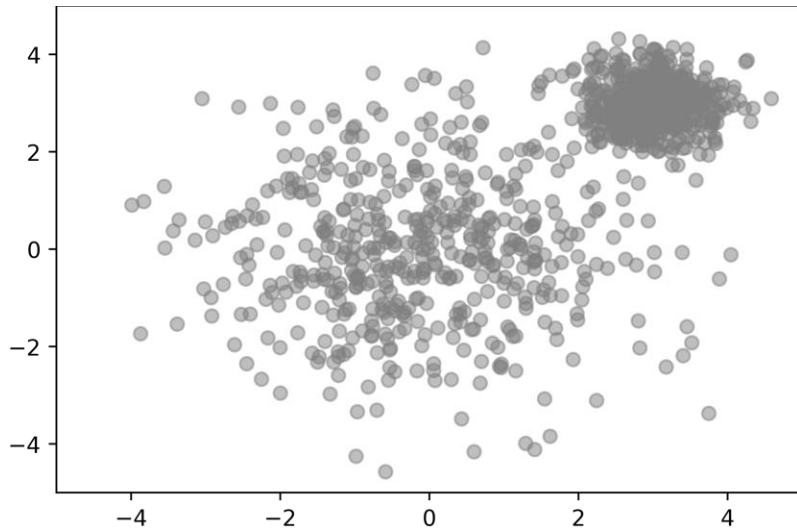


Batch Normalization

Batch Normalization

- `tf.keras.layers.BatchNormalization`
- Improves training
- Often speeds up training

$$x_{t+1} = \frac{x_t - \text{mean}(x_t)}{\sqrt{\text{std}(x_t) + \varepsilon}} * \gamma + \beta$$

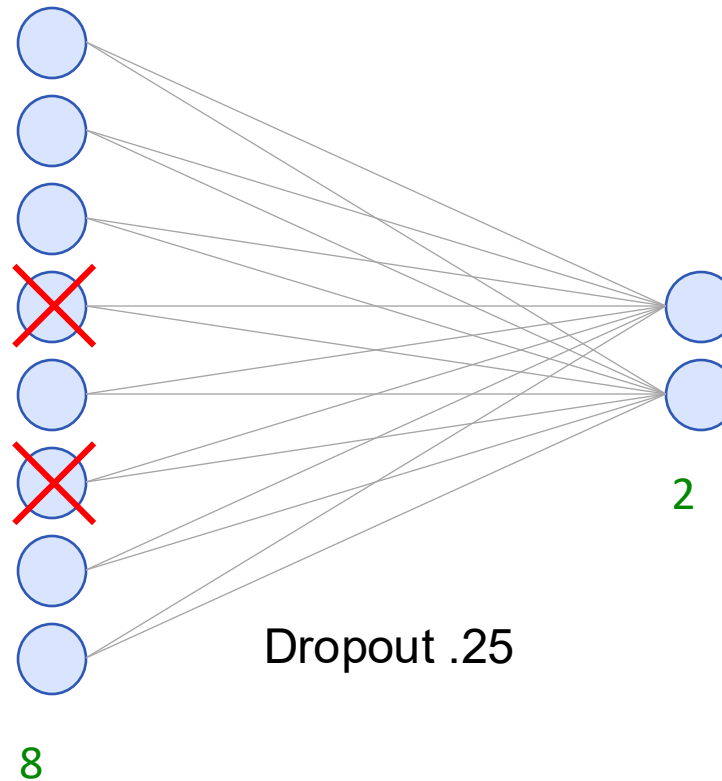


Ioffe and Szegedy 2015 <https://arxiv.org/abs/1502.03167>

Dropout

Dropout

- Reduces overfitting



Dropout layer reduces the likelihood to rely on one single “feature” as sometimes its not active.

What can we do now?

Example Model

```
model = Sequential()

model.add(InputLayer((28,28,1), name = "InputLayer"))

model.add(Conv2D(filters=128, kernel_size=(3,3), activation='relu', padding='same'))

model.add(MaxPool2D())

model.add(Dropout(rate=0.25))

model.add(Conv2D(64, kernel_size=(3,3), activation='relu', padding='same'))

model.add(MaxPool2D())

model.add(BatchNormalization())

model.add(Flatten())

model.add(Dense(128, name = "HiddenLayer1", activation='relu'))

model.add(Dense(64, name = "HiddenLayer2", activation='relu'))

model.add(Dense(2, name = "OutputLayer", activation=softmax'))
```

new layer Flatten
needed after Conv2D to
use a Dense next –
similar to reshape



Conclusion

Layers

- Convolutional Neural Network (CNN)
- Max Pooling
- Batch Normalization
- Dropout

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer





Artificial Intelligence in Interactive Systems

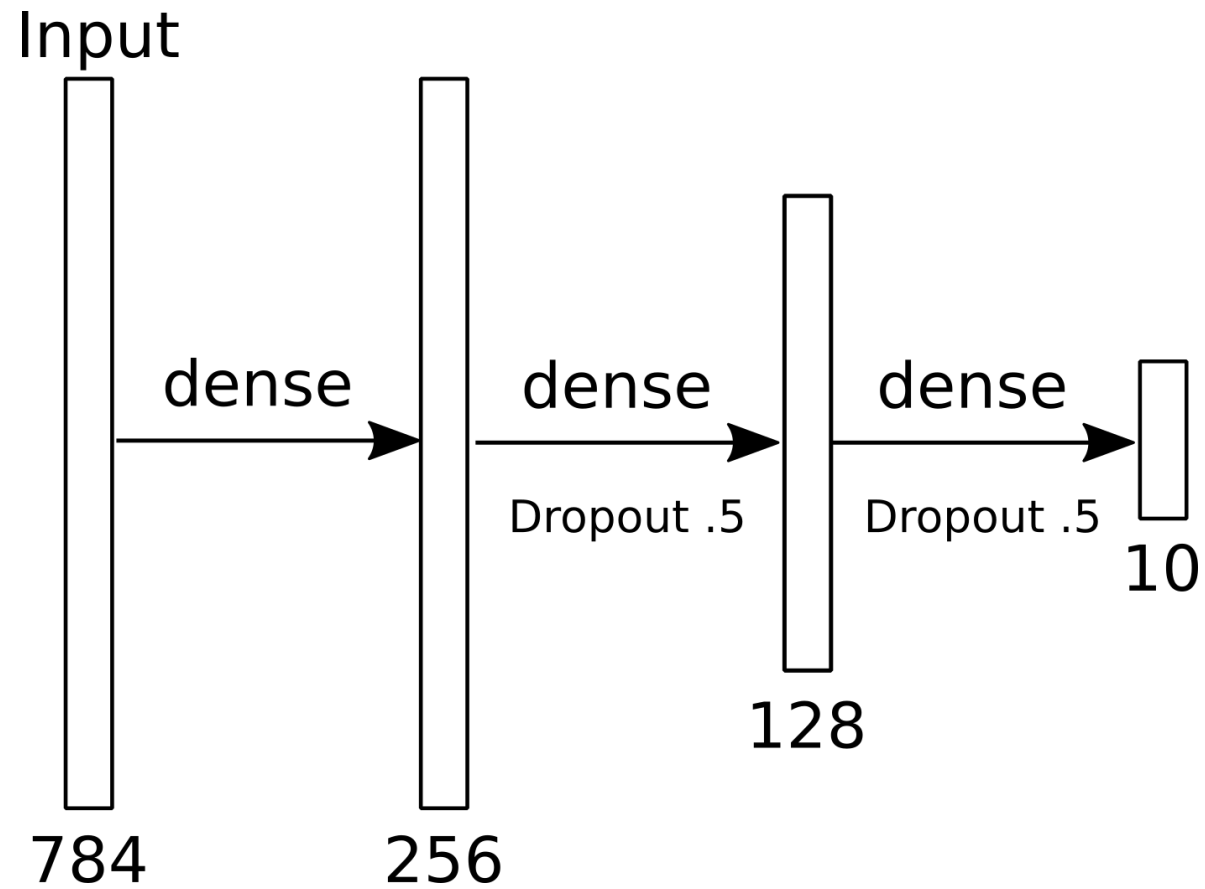
Optimizer and Hyperparameter

Impact of Hyperparameter on the Models

MNIST data as showcase



Test Model



Optimizer

Optimizer

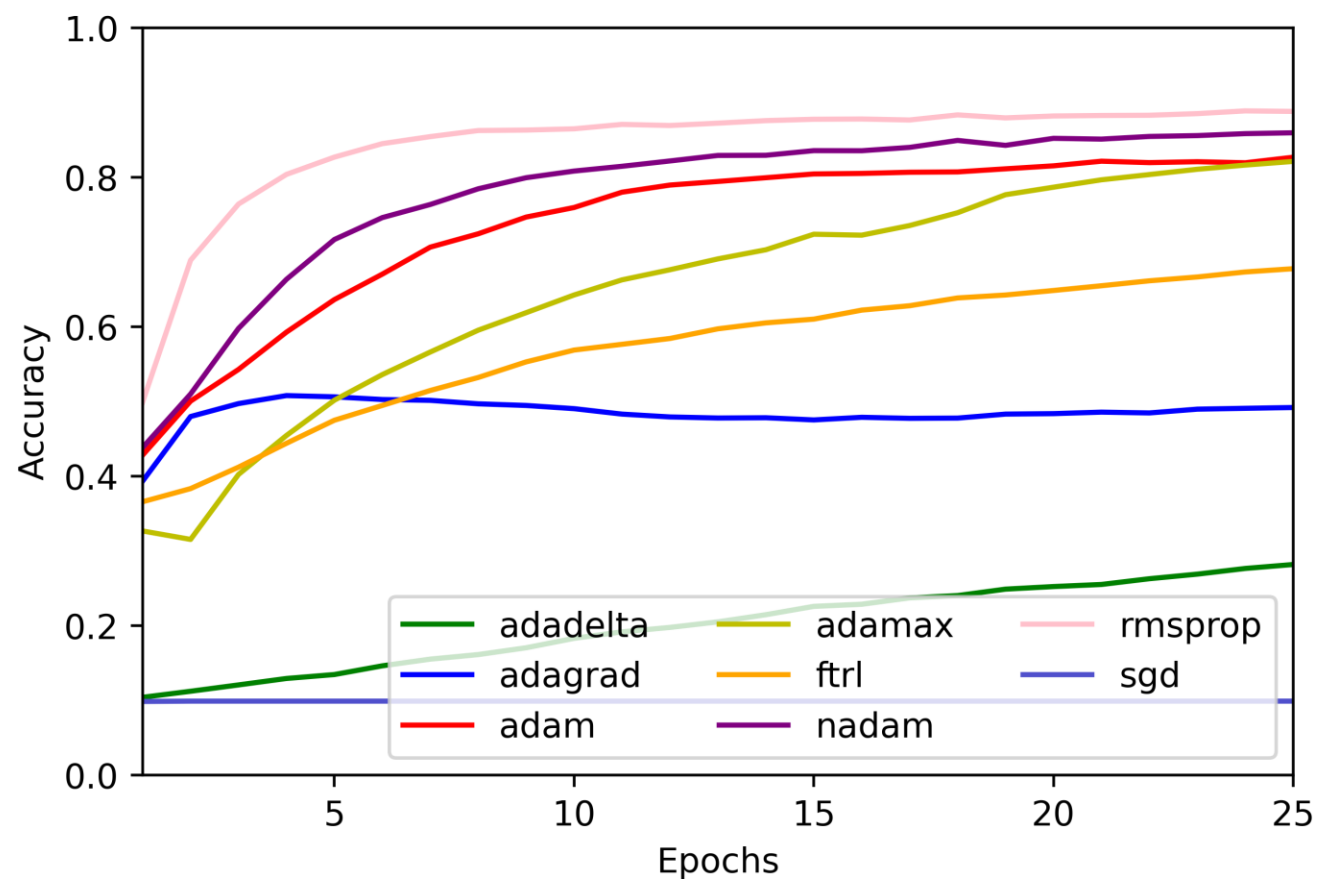
The optimizer is an algorithm that decided on how to adjust the weights and biases during backpropagation. This is done by minimizing the cost described by the loss function.

Optimizer

- Adadelta (2012 <https://arxiv.org/abs/1212.5701>)
- Adagrad (2011 <https://www.jmlr.org/papers/volume12/duchi11a/duchi11a.pdf>)
- Adam (v1: 2014 v9: 2017 <https://arxiv.org/abs/1412.6980>)
- Adamax (version of Adam <https://arxiv.org/abs/1412.6980>)
- FTRL (2013 <https://research.google.com/pubs/archive/41159.pdf>)
- Nadam (2015 http://cs229.stanford.edu/proj2015/054_report.pdf)
- Root Mean Square Propagation (RMSprop) (2012 http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf)
- Stochastic gradient descent (SGD)

Optimizer Test

With default settings



2 dense layers with dropout tested on a subset of the MNIST dataset

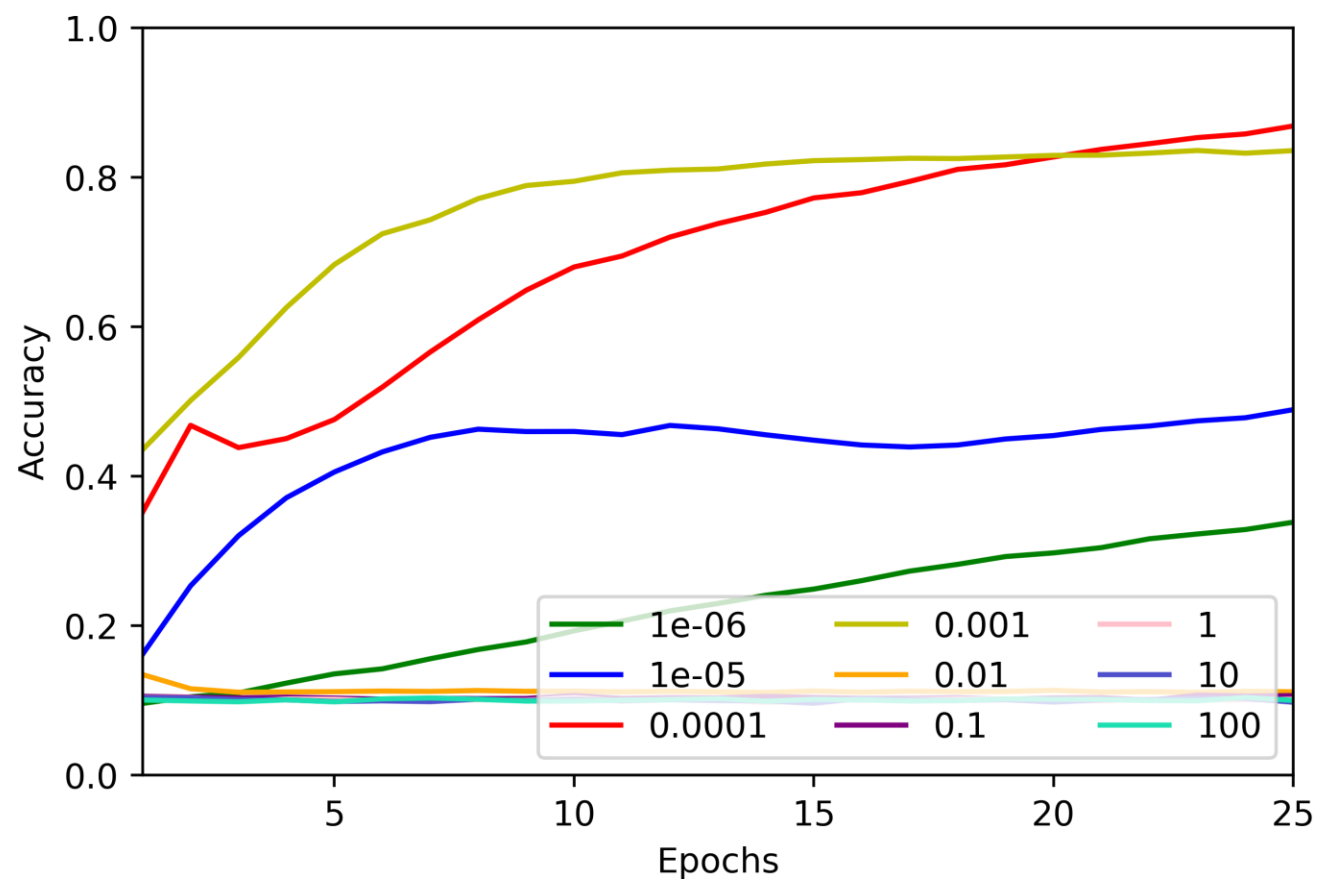
Learning Rate

Learning Rate

The learning rate is the rate on how much the optimizer adjusts the weights and biases in each backpropagation step.

Learning Rate Test

Effect of the Initial Learning Rate Using Adam



2 dense layers with dropout tested on a subset of the MNIST dataset

Adaptive Learning Rate

- Learning rate decay over time
- Learning rate decay on no change (plateau)
- Use of “*momentum*” to accelerate training

Loss Function

Loss Function

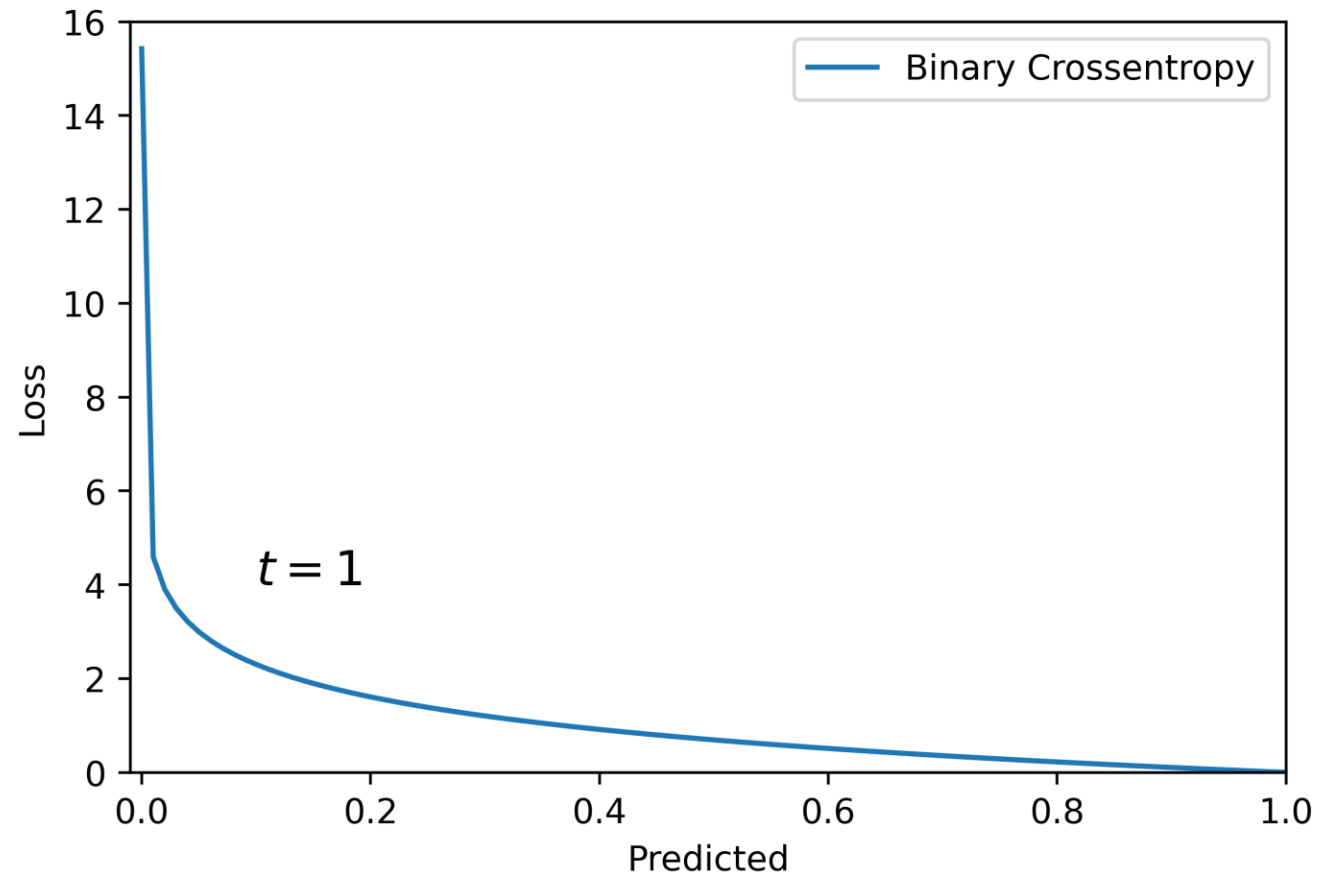
A loss function (cost function) is a function to determine how different the prediction is with respect to the ground truth. The optimizer's goal is to minimize the loss.

Loss Functions

- Discrete Data
 - Binary Cross Entropy
 - Categorical Cross Entropy
 - Hinge loss
- Continues Data
 - Mean Absolute Error (MAE) (also know as L1 Loss)
 - Mean Squared Error (MSE) (also know as L2 Loss)
 - Mean Squared Logarithmic Error

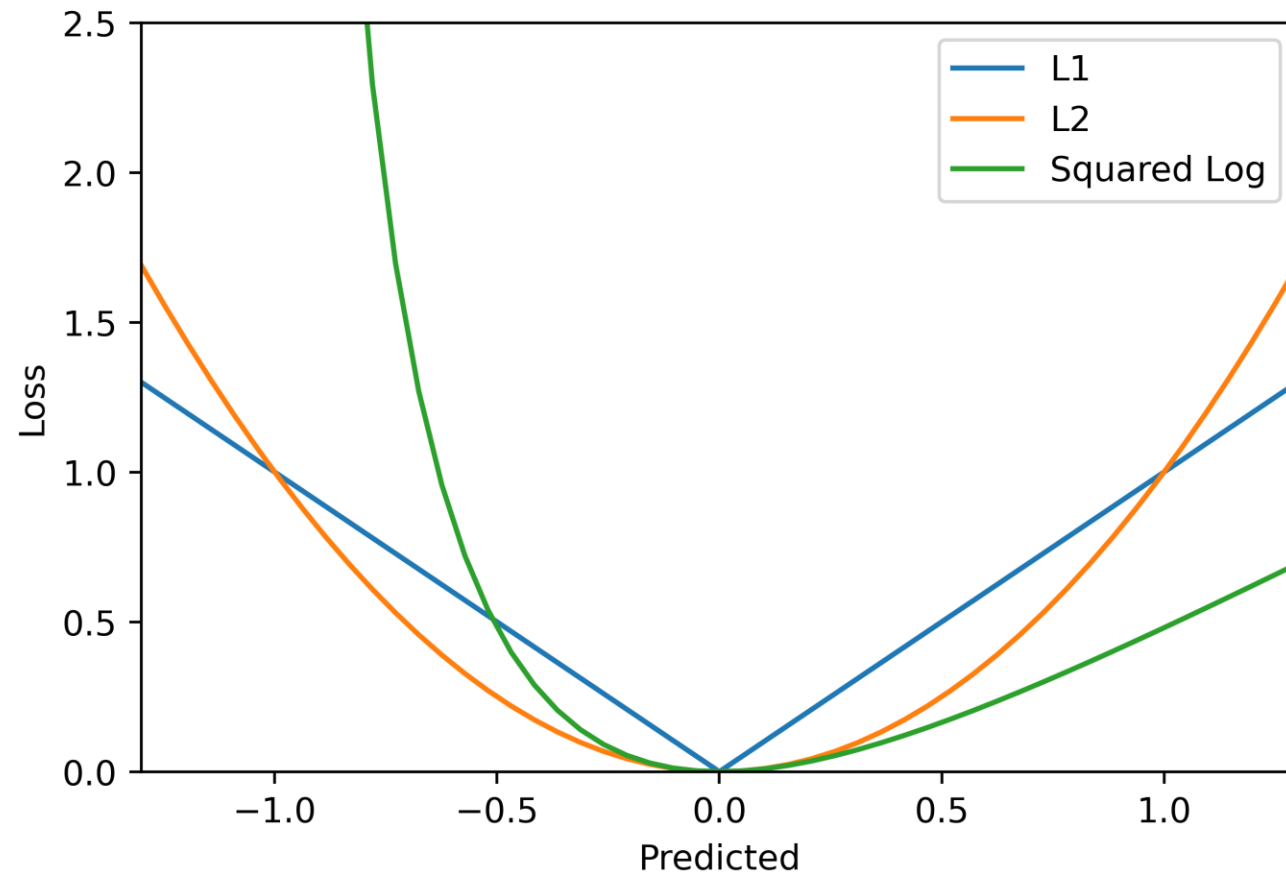
Loss Functions

Classification



Loss Functions

Regression



Loss Functions

Advanced Loss

- Pixel-wise loss
- Feature matching (FM), e.g. [1]

$$loss = \lambda_1 * MSE + \lambda_2 * FM$$

In the image domain, such approaches are used to counteract blurry images.

[1] Florian Schroff, Dmitry Kalenichenko, James Philbin; Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 815-823

Conclusion

Layers

- Optimizer
- Learning Rate
- Adaptive Learning Rate
- Loss Function

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer



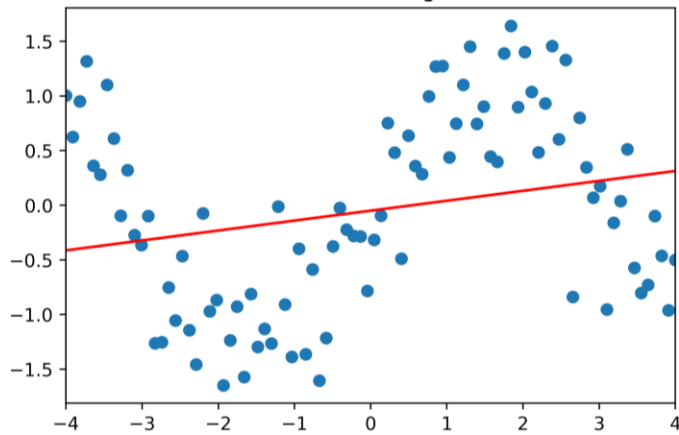


Artificial Intelligence in Interactive Systems

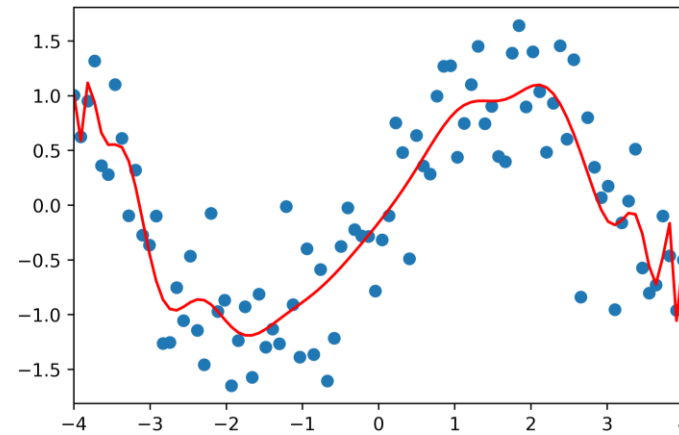
Over- and Underfitting

Over- and Underfitting

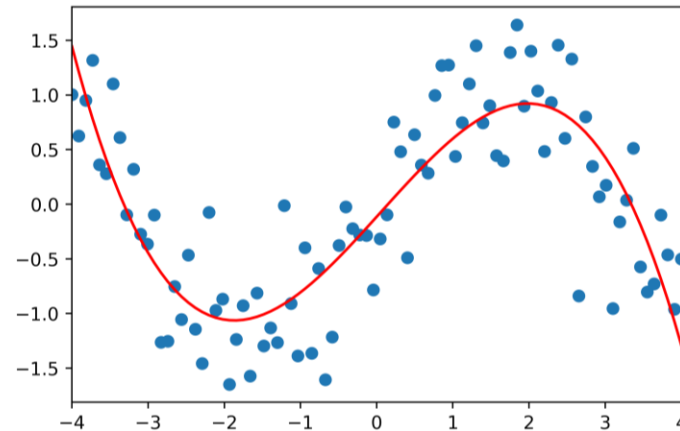
Underfitting



Overfitting

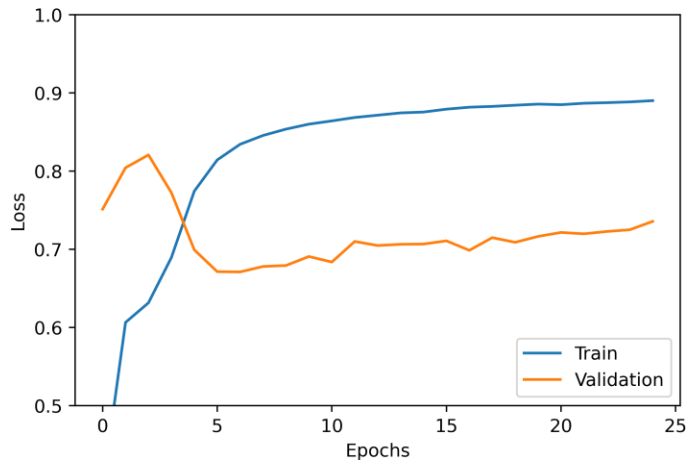


Good Fit

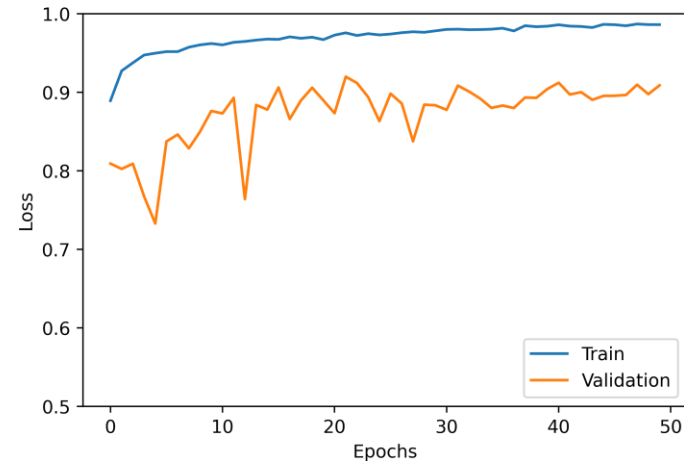


Over- and Underfitting

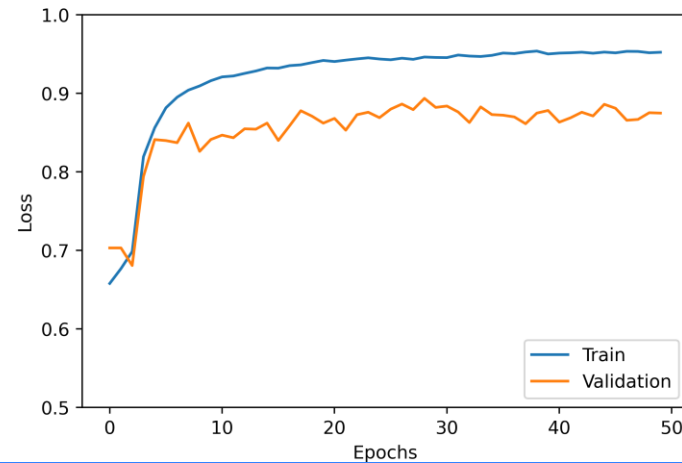
Underfitting



Overfitting



“Good” Fit



Conclusion

Over- and Underfitting

- Overfitting
- Underfitting

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer

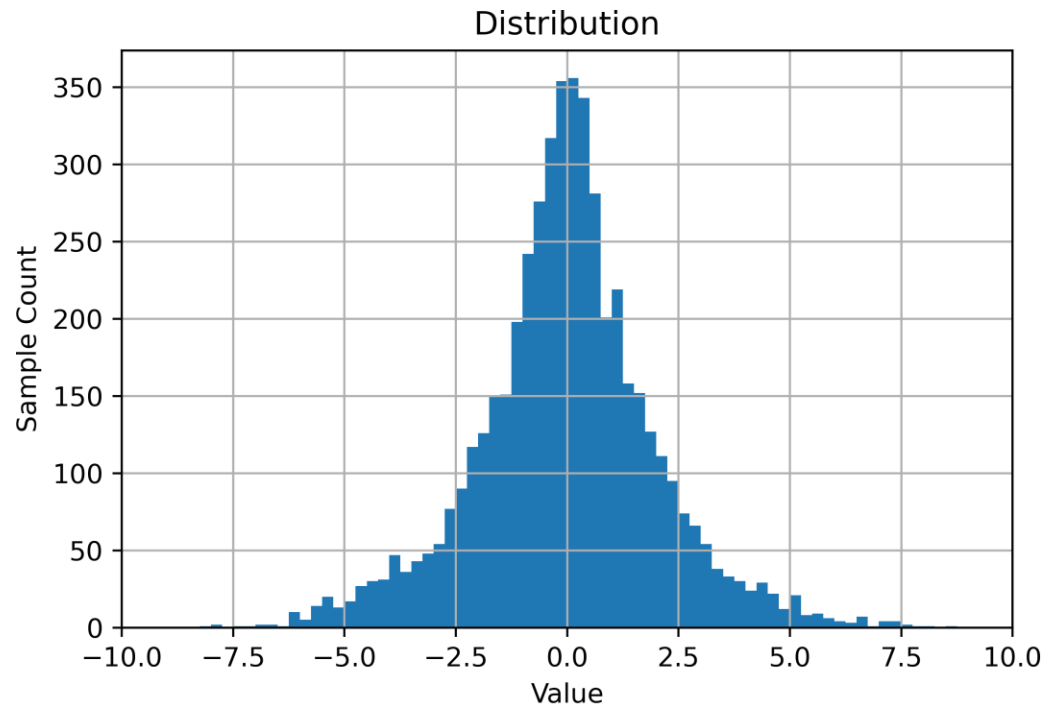




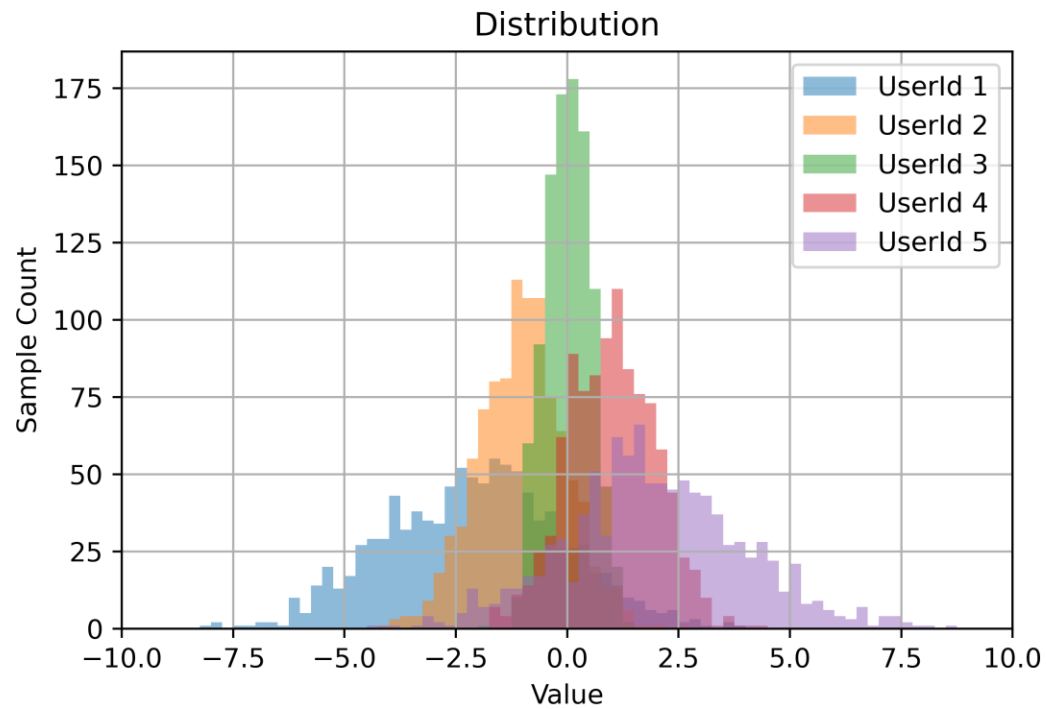
Artificial Intelligence in Interactive Systems

Training-Validation-Test Split

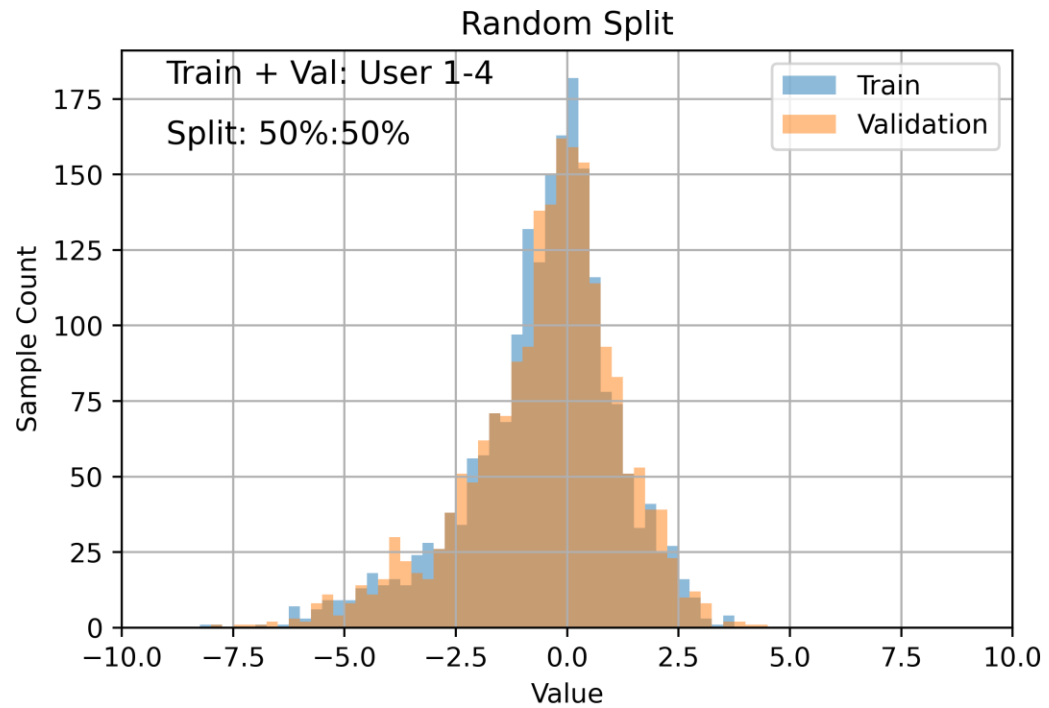
Training-Validation-Test Split



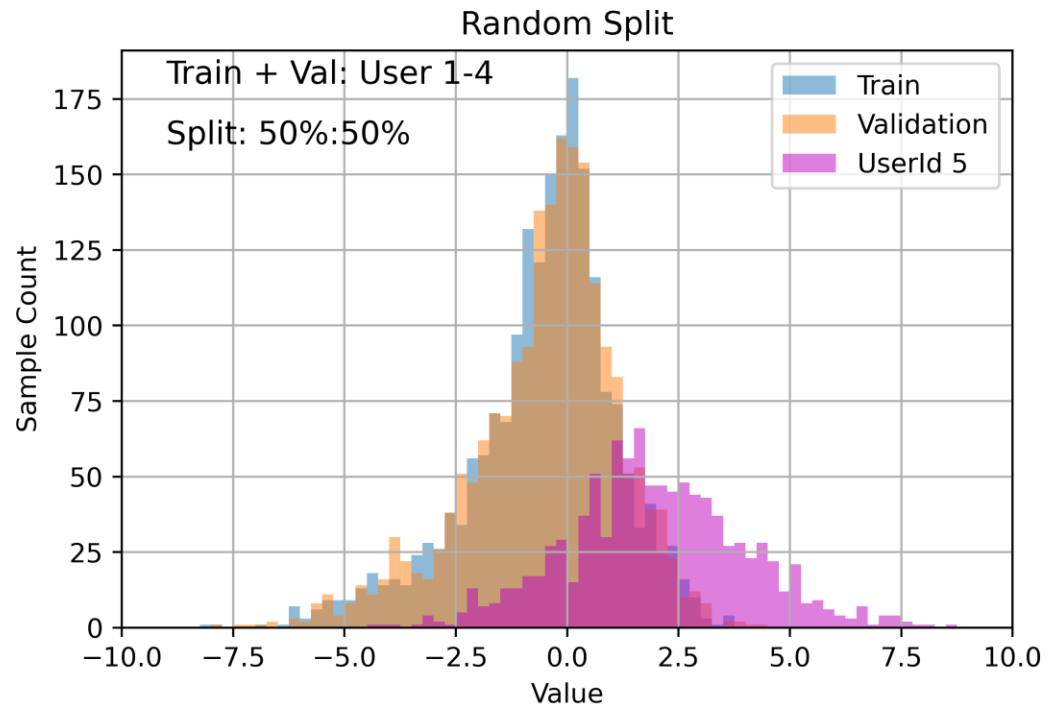
Training-Validation-Test Split



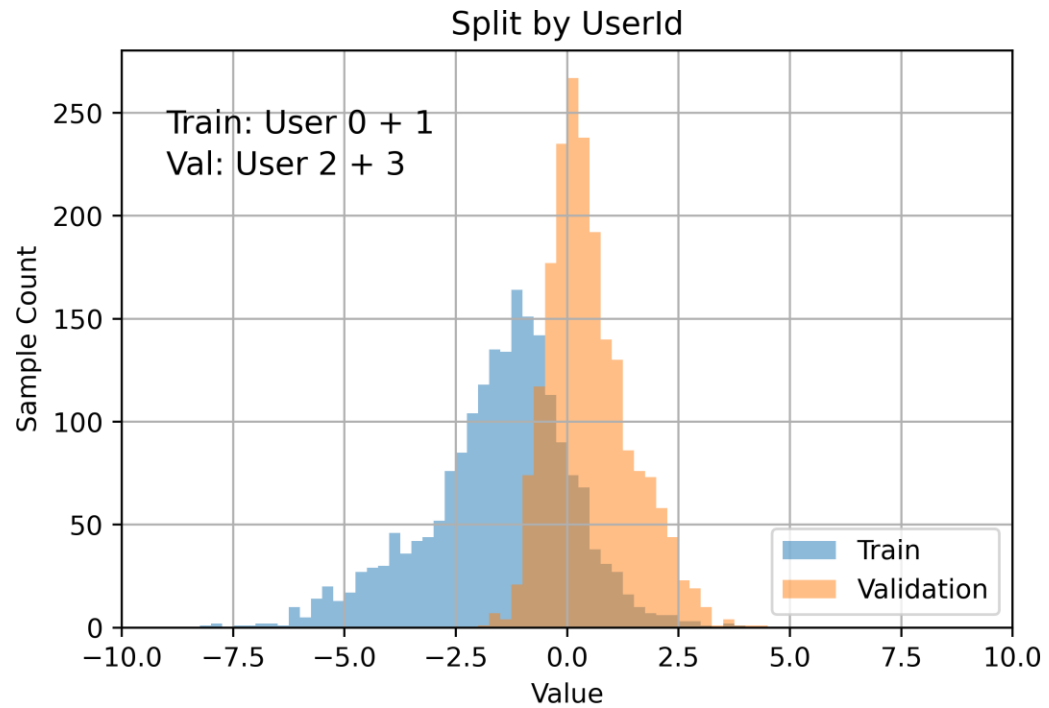
Random Split



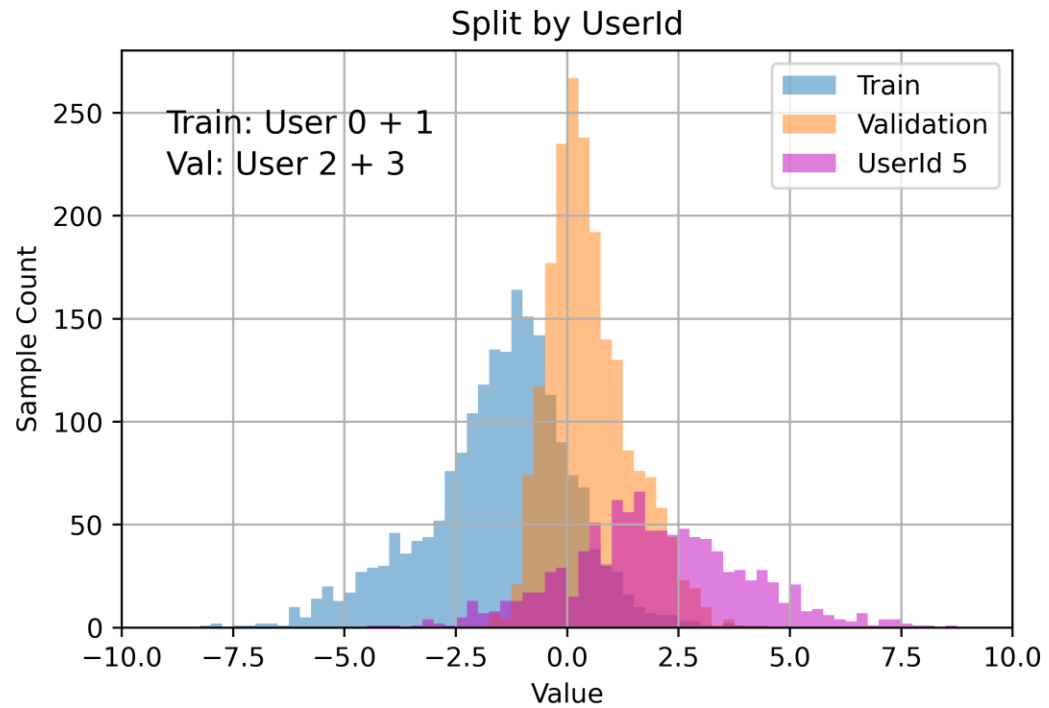
Random Split



Participant Wise Split



Participant Wise Split



Cross-Validation

- Leave- k -Out Cross-Validation
- Leave-One-Group-Out Cross-Validation
- Leave-One-Participant-Out Cross-Validation

Conclusion

Training-Validation-Test Split

- Random split
- Participant wise split
- Leave-One-Participant-Out Cross-Validation

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer





Artificial Intelligence in Interactive Systems

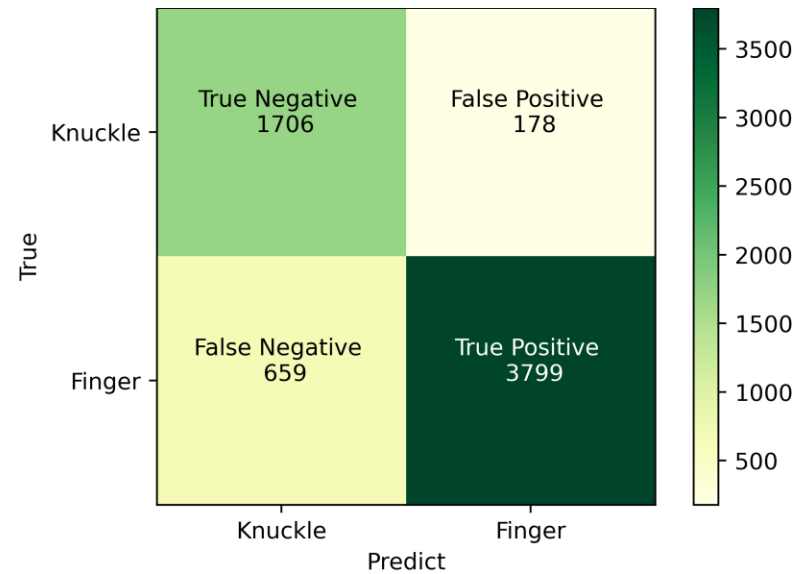
Evaluation Metrics

Evaluation Metrics

- Mean Squared Error (MSE)
- Mean Squared Logarithmic Error
- Categorical Cross Entropy
- Loss Functions

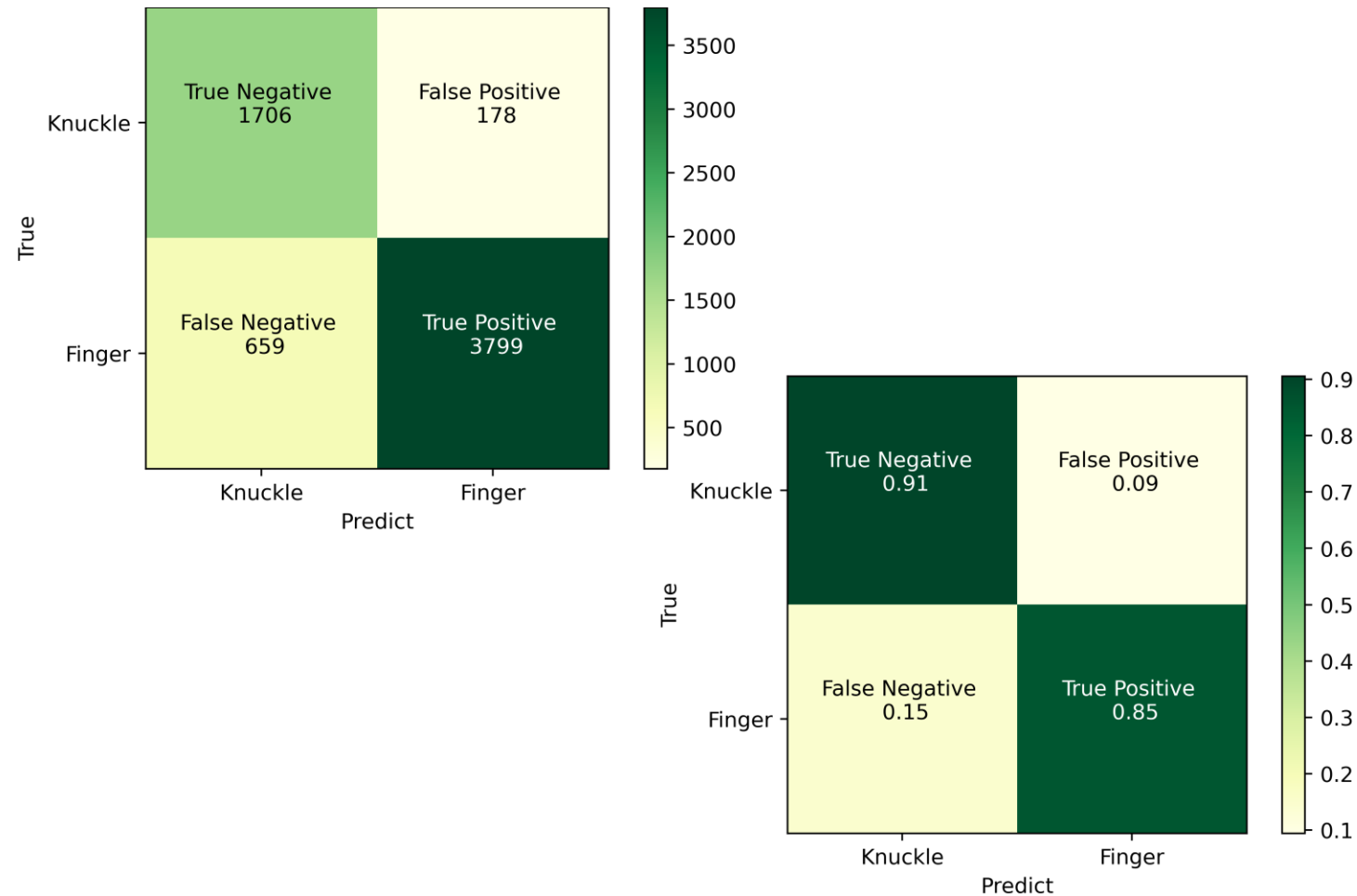
Confusion Matrix

Binary Classification



Confusion Matrix

Binary Classification



Precision, Recall, and F1 score

$$\textit{Precision} = \frac{TP}{TP + FP}$$

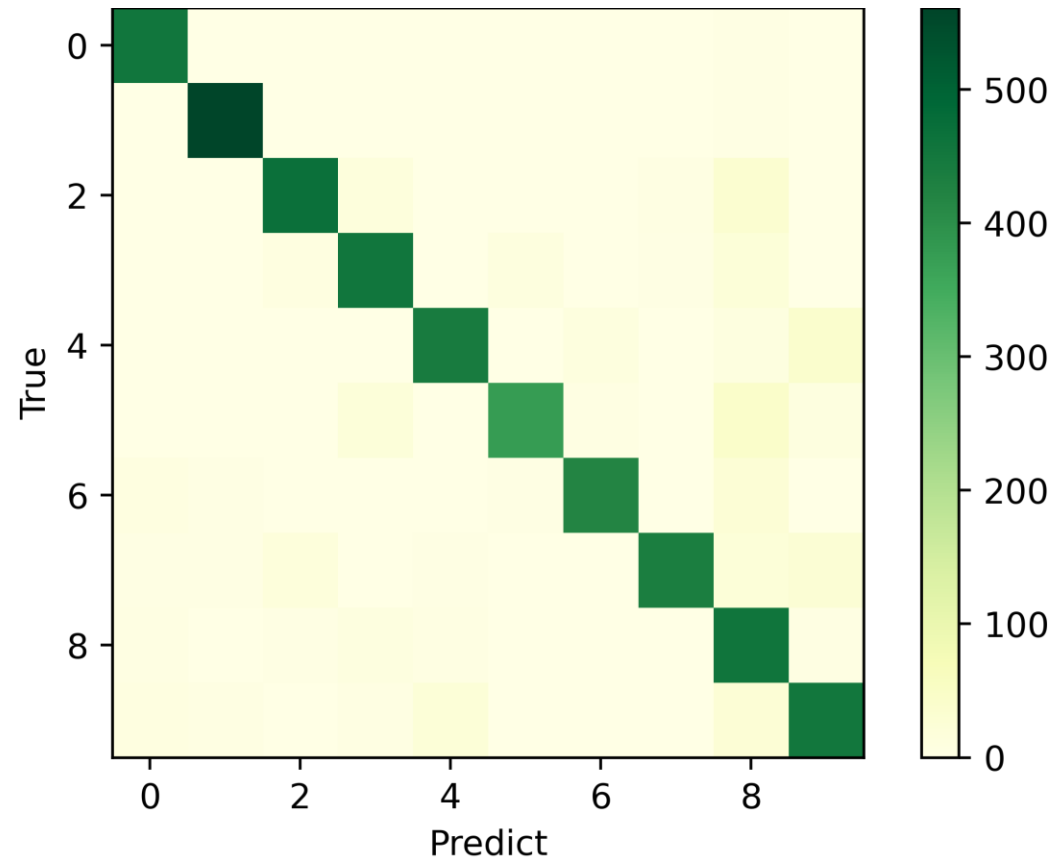
$$\textit{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 * \frac{\textit{Precision} * \textit{Recall}}{\textit{Precision} + \textit{Recall}}$$

$$\textit{Accuracy} = \frac{TP + TN}{TN + FP + TN + TP}$$

Confusion Matrix

Multi-Class Classification



Conclusion

Evaluation Metrics

- Precision, Recall, and F1 score
- Confusion Matrix

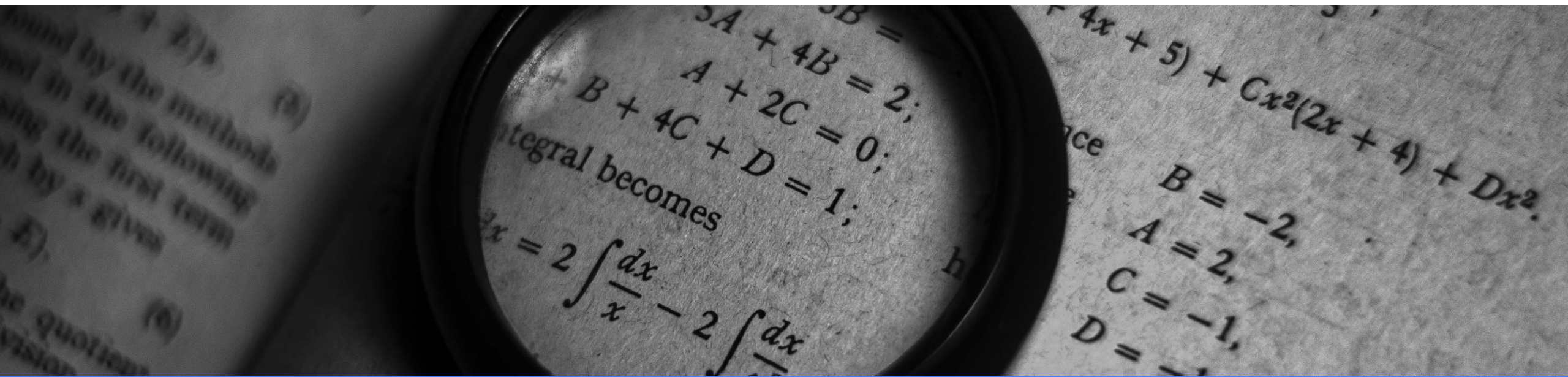
License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer





Artificial Intelligence in Interactive Systems

Hyperparameter Tuning

Hyperparameter Tuning

- Model structure
- Loss function
- Optimizer
- Learning Rate

- Dropout
- Batch Normalization

Random Search

- Time-consuming
- Not effective for large parameter spaces

Randomly testing all parameter.

- Use Case: Hyperspace exploration

Grid Search

- Traditional ML:
 - `sklearn.model_selection.GridSearchCV`

Grid Search is the search through a manually specified set of parameters.

- Time consuming
- Not effective for Deep NN due to the large parameter space
- “Partial” Grid Search can be effective

Trial and Error

Trial and Error is a search in which new hyperparameters are tested until a "good" model is found. The hyperparameters are picked by analyzing the failed runs.

- Common approach for deep NN
- The initial seed can be random or based on prior knowledge

Network Growing and Network Pruning

Network Growing and Network Pruning is used to determine how many hidden neurons are “optimal.” For Starting with a small model and adding neurons until the good solution is found.

- Network Growing
 - Less Time-Intensive
 - Smaller molds
- Network Pruning
 - Time-Intensive
 - Large molds

Du, K. L., & Swamy, M. N. (2006). *Neural networks in a softcomputing framework*. Springer Science & Business Media.

Domain Knowledge Based Method

Limiting the search space by using prior knowledge helps especially in the early steps when still exploring the vast parameter space.

- What worked before?
- What worked for others?

Early Stopping

Stops training when no improvement happens over a certain period.

- Saves a lot of time
- `tf.keras.callbacks.EarlyStopping`

Conclusion

Hyperparameter Tuning

- Random Search
 - Grid Search
 - Trial and Error
 - Network Growing and Network Pruning
 - Domain Knowledge Based Method
 - Early Stopping
-
- Combining different strategies
 - Keep track of all trained models
 - Parallel and sequential training

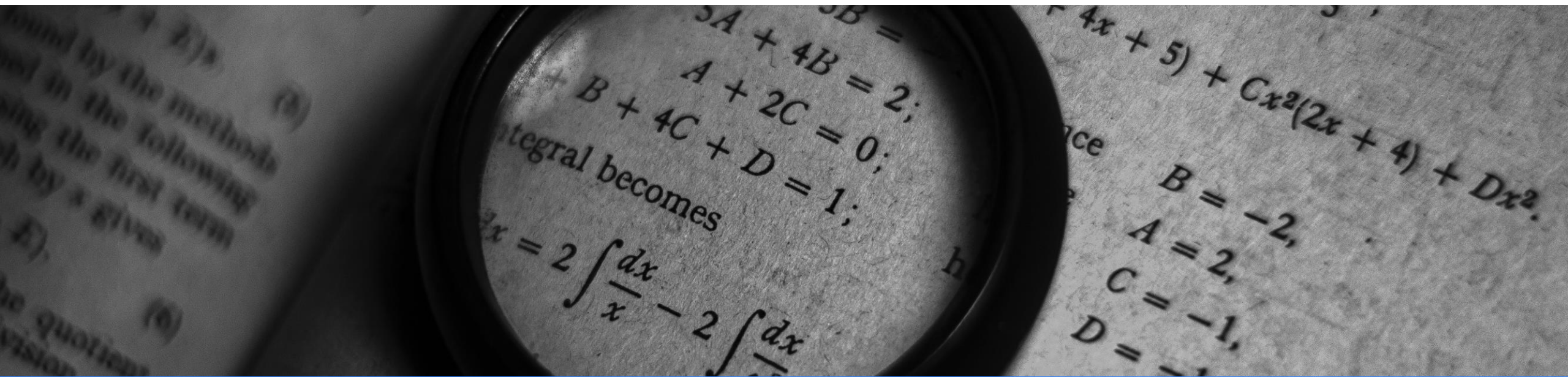
License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer





Artificial Intelligence in Interactive Systems

Pre-Trained Models

Pre-Trained Models

Larger general models help to get started training and reduce training time.

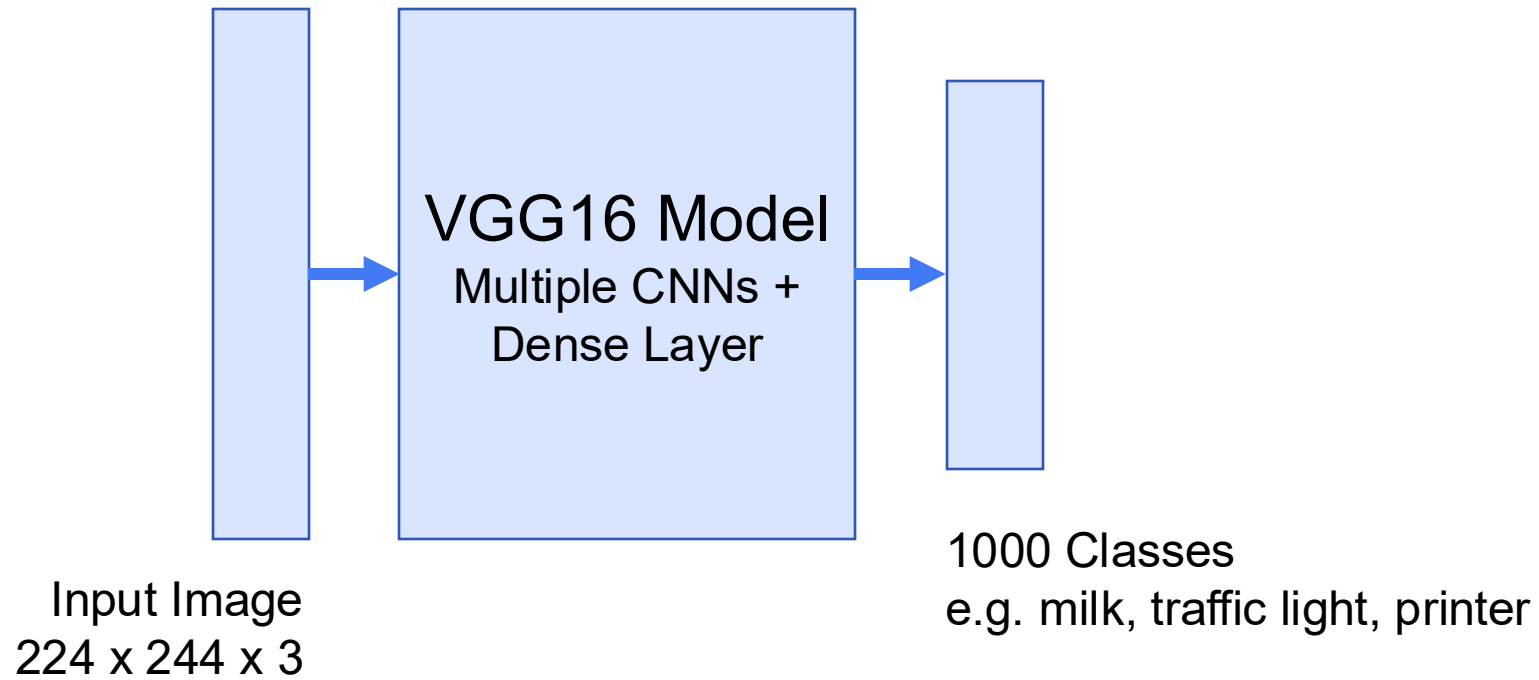
- Good starting point – domain knowledge
- Reduces training time
- Reduces computational complexity
- Allows to train with less data – “fine tuning”

Pre-Trained Models

- Reducing the output classes
 - 1000 -> 5
- Used as feature extraction
 - Fitting the model to new classes

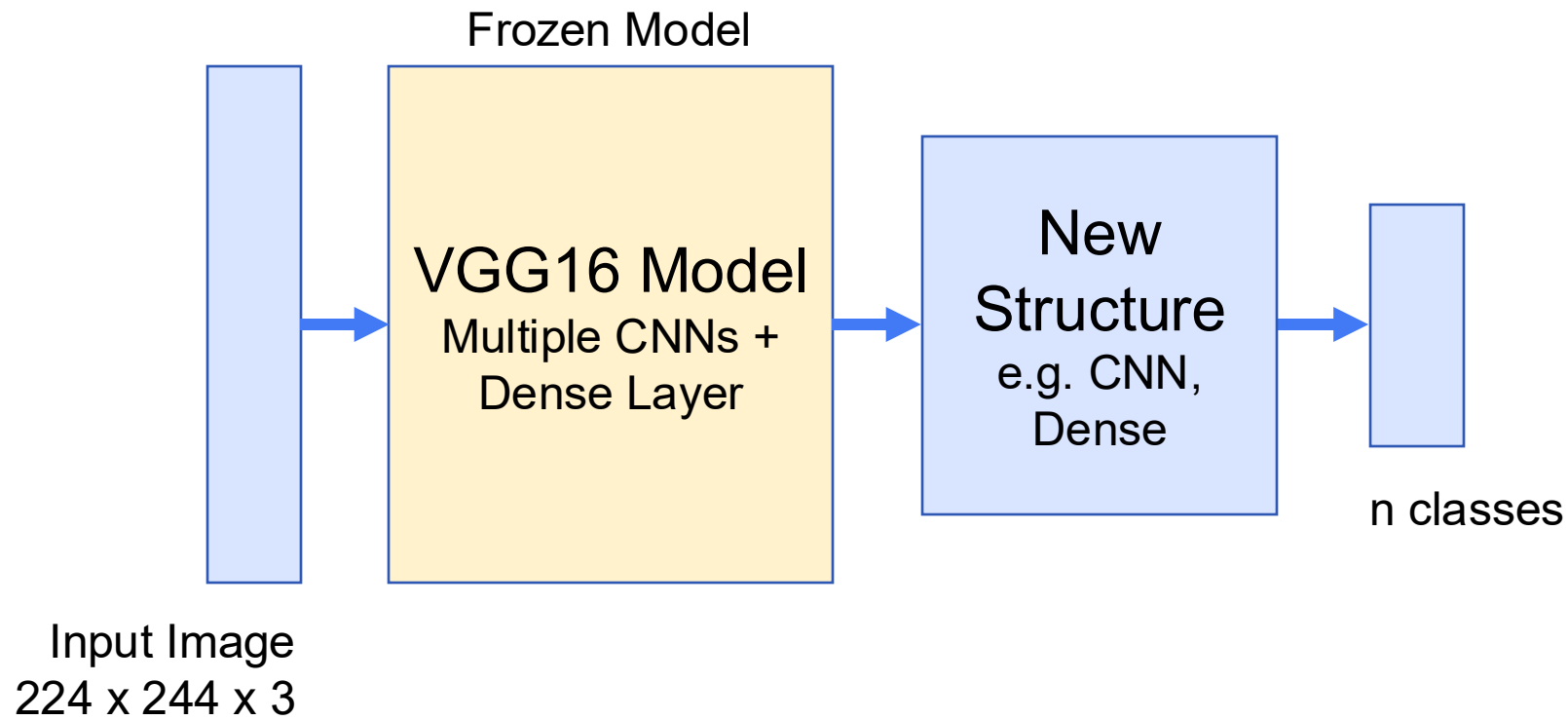
Pre-Trained Models

VGG16 Example



Pre-Trained Models

VGG16 Frozen Model + New Structure



Example Model

```
model = Sequential()  
model.add(Input((224, 224, 3), name="Input"))  
  
vgg = tf.keras.applications.VGG16( weights='imagenet', include_top=False)  
model.add(vgg)  
  
model.add(Flatten())  
model.add(Dense(1024, activation='relu'))  
model.add(Dropout (0.5))  
model.add(Dense, 5, activation='softmax'))
```

Models

- Various Model in TensorFlow
 - https://www.tensorflow.org/api_docs/python/tf/keras/applications
- All models can be included like this
 - Models from your own work
 - Models from prior research
 - Any open source model

Conclusion

Pre-Trained Models

- Feature reduction using pre-trained models
- Any trained model can act as such a “layer”

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer

